

ARM64 vs x86 Synchronization: What the Numbers Actually Say

Github Repository: <https://github.com/DjFigueiredo/MarketDataPipeline>

Introduction

Recently I rotated through an engineering team at Caterpillar Inc. where I developed low-level software for Telematics Engine Control Units on processors running Linux. That was a step outside my normal path, I had focused on low-level software for RTOS machines for a majority of my time at Cat.

Moving into lower-latency systems work raised a question I couldn't answer from documentation alone: how much does ISA-level **memory** ordering actually cost, and does the ARM64/x86 gap matter in practice?

On ARMv8 the compiler must emit ordering-bearing instructions for `memory_order_acquire` and `memory_order_release`. On my hardware those are `ldapr` and `stlrr`, confirmed by disassembly below, with semantics defined in the ARM Architecture Reference Manual. Intel's SDM Vol. 3A §8 describes x86's TSO model: the hardware already enforces those ordering constraints, so acquire/release compile to plain loads and stores. Different contracts. I wanted the measured delta, not the architectural description.

I ran three benchmarks on two physical machines: a MacBook Pro M3 Pro (ARM64, macOS) and a Dell Precision 3640 running Ubuntu bare metal (Intel 10th Gen Comet Lake, x86-64). Bare metal matters, a hypervisor remaps vCPUs between physical cores and pollutes both pinning assumptions and tail latency.

What follows are the results from a counter contention ladder (V1-V4), a pingpong acquire/release round-trip, and an SPSC queue throughput test, with the mechanism behind each number.

Hardware Specifications

	MacBook Pro (ARM64)	Dell Precision 3640 (x86-64)
CPU	Apple M3 Pro (6P + 6E cores, 12 total)	Intel i9-10900, 10C/20T, Comet Lake
Architecture	ARM64	x86-64
OS	macOS Sequoia 15.7.3	Ubuntu 24.04 (bare metal)
RAM	36 GB	32 GB
Kernel	24.6.0	6.8.0-28-generic
Cache Line	128 bytes (measured via <code>sysctl</code>)	64 bytes
L1d / L1i	P-core: 128 KiB / 192 KiB; E-core: 64 KiB / 128 KiB (measured; no published spec)	32 KiB / 32 KiB per core
L2	P-core: 16 MiB per cluster; E-core: 4 MiB (measured; no published spec)	256 KiB per core
L3	No conventional L3; a system-level cache serves the role (size unpublished; inferred)	20 MiB (shared)
Timer	<code>mach_absolute_time</code> , 24 MHz, ~42 ns/tick	<code>rdtscp</code> , serialized cycle counter
Thread Pinning	<code>pthread_set_qos_class_self_np</code> (QoS hint only, no hard affinity API)	<code>pthread_setaffinity_np</code>
Core Isolation	N/A	<code>isolcpus=0,1,2,3</code> + <code>nohz_full</code> + <code>rcu_nocbs</code>

How I Measured

Timing

Both platforms use the highest-resolution timer available without OS involvement on the hot path.

Linux (x86-64): `rdtscp`, Intel's serializing timestamp counter. Serializing here means `rdtscp` waits for all prior instructions to complete before reading the counter. It does not stop later instructions from starting early, so it bounds the start of a measured region tightly, and at these interval lengths the end is bounded well enough that no trailing fence was added. The TSC is invariant on the i9-10900: it ticks at a fixed frequency regardless of CPU boost state or C-state transitions. That frequency was not assumed from the spec sheet; it was calibrated at runtime by correlating `rdtscp` deltas against `CLOCK_MONOTONIC_RAW` over a 200 ms `nanosleep` interval. Measured TSC frequency: **2.808 GHz**. All cycle counts in this document convert to nanoseconds using that calibrated figure.

macOS (ARM64): `mach_absolute_time()` converted via `mach_timebase_info`, Apple's recommended high-resolution timer. On M3 Pro this ticks at 24 MHz (~41.7 ns per tick). This is a hard resolution floor: any latency value reported on Mac is quantized to the nearest tick boundary. Throughput measurements are reliable; latency percentiles are tick counts, not exact nanoseconds. This quantization is called out explicitly wherever it affects interpretation.

CPU Frequency and Stability

Linux: CPU governor was set to `performance` via `cpupower frequency-set -g performance`, and `intel_pstate` minimum performance was pinned to 100% (`/sys/devices/system/cpu/intel_pstate/min_perf_pct = 100`) to prevent idle clock-down between benchmark phases. Actual operating frequency was verified via `perf stat` cycles divided by wall time; `scaling_cur_freq` on this `intel_pstate` system reflects idle P-state snapshots, not the turbo frequency achieved during the workload. Turbo boost confirmed enabled (`no_turbo = 0`).

macOS: No equivalent frequency-pinning API exists.

`pthread_set_qos_class_self_np(QOS_CLASS_USER_INTERACTIVE, 0)` was used as a scheduling hint. Low Power Mode was disabled for all runs. On Apple Silicon this mode caps P-core frequency and produces 2-3x slowdowns that are indistinguishable from code regressions without an explicit A/B check. `macmon` was used to monitor cluster frequencies and confirm the M3 Pro was not throttling during benchmark runs.

Thread Pinning and Core Isolation

Linux: `pthread_setaffinity_np` pins each thread to a specific physical core. Cores 0-3 were removed from the kernel scheduler entirely via kernel boot parameters (`isolcpus=0,1,2,3 nohz_full=0,1,2,3 rcu_nocbs=0,1,2,3`). This eliminates timer interrupts and RCU callbacks on the benchmark cores. The pingpong tails were collected under this configuration, and the Mac/Linux tail comparison in that section reflects the asymmetry.

macOS: Hard affinity is not available; `pthread_setaffinity_np` does not exist on macOS. The QoS hint used here was tested and did not measurably tighten variance on contended variants. The macOS results therefore carry higher tail sensitivity to OS scheduler noise than the Linux results. This asymmetry is the primary reason Mac and Linux p99.9 values are not directly comparable.

Hardware Counter Collection

Linux: `perf stat` wraps the benchmark binary to collect aggregate hardware counters (LLC load misses, instructions, cycles) without modifying the binary. Counter collection and timing runs are kept separate, so the `perf` overhead does not appear in the latency numbers reported in the results tables.

```
perf stat -e cycles,instructions,LLC-load-misses,LLC-loads ./pingpong_bench
```

macOS: `xctrace` with the "CPU Counters" template is the equivalent; it accesses Apple Silicon PMU counters from the terminal without requiring the Xcode GUI. Like `perf stat`, it is run separately from the timing benchmark to avoid overhead contaminating the latency results.

```
xctrace record --template "HW_Counters" --launch -- ./pingpong_bench --  
output counters.trace
```

The resulting `.trace` file is inspected in Instruments. Note that Apple Silicon PMU counter names do not map 1:1 to Intel's (e.g., there is no direct `LLC-load-misses` equivalent by that name). The specific counters used and their Apple Silicon labels are noted alongside each result where hardware counter data is cited.

Instruction Verification (macOS)

To verify that the compiler emitted the expected instructions on Apple Silicon and that no unintended barriers or retry loops appeared in the hot paths, all three binaries were disassembled using:

```
xcrun llvm-objdump -d --mcpu=apple-m3 ./build/bench/<binary> >
notes/external/disasm_mac.txt
```

The resulting output was grepped for the instructions that the paper's analysis depends on. Key findings:

ldadd: ARMv8.1 LSE atomic add. `counter_bench`'s `fetch_add(memory_order_relaxed)` compiles to a single `ldadd` instruction, not an `ldxr/stxr` exclusive-monitor loop. `ldadd` atomically reads a memory location, adds a value, and writes the result back in one hardware operation. Like x86's `lock xadd`, it never fails and has no software retry path. M3 Pro supports ARMv8.1 Large System Extensions, so the compiler emits this rather than the older exclusive-monitor pattern.

ldapr: Load-Acquire RCpc. `pingpong_bench`'s `memory_order_acquire` load emits `ldapr`, not the older `ldar`. RCPC (Release Consistency, Processor Consistent) provides acquire semantics against prior stores without the full cost of `ldar`'s TSO-equivalent barrier. This is the ARMv8.3 RCPC extension, available on Apple Silicon.

stlr: Store-Release. `memory_order_release` stores emit `stlr` across all benchmarks. This is the base ARMv8 store-release instruction; it prevents the store from being observed before any prior memory accesses.

yield: Spin hint. The spin-wait loop in `pingpong_bench` emits the `yield` instruction (the ARM equivalent of x86 `_mm_pause`). It hints to the CPU that the core is in a spin-wait, allowing the hardware to reduce power and back off resource contention. Unlike `_mm_pause` on x86, `yield` carries no guaranteed pipeline stall duration; its effect is microarchitecture-dependent and measurably weaker.

No dmb/dsb/isb. No explicit memory barrier instructions appear in any hot path. All ordering is carried by the `ldapr/stlr` instructions themselves, consistent with the ARM ARM's specification for acquire/release on ARMv8.3+.

No ldxr/stxr. The exclusive-monitor loop does not appear anywhere in the counter or pingpong hot paths. This corrects an earlier assumption in the analysis; see General Findings below.

Sampling Strategy

All benchmarks use an in-process best-of-N harness rather than re-launching the binary N times. Relaunching introduces inter-run OS and thermal entropy; in-process sampling amortizes cold-start cost across one binary invocation. Counter uses a best AND average of 20, pingpong was re-ran 10 times, SPSC uses best-of-10 with 10 internal iterations. Raw samples are sorted and percentiles are computed directly, with no smoothing or outlier removal.

Testing

We have three benchmarks we are going to look at. Counter Benchmark, Pingpong Benchmark, and SPSC Benchmark. Below you will find details and test results for each benchmark.

Counter Benchmark (V1-V4)

Test Variations

The first series of tests is 4 variations of incrementing a counter 10 million times across 4 threads. Each variation removes a weakness the previous variation had. That is we start with a lock, then move to atomicity, then multiple atomic variables, followed by independent cachelines.

1. Variant 1 Mutex: Increment the counter using a basic lock/unlock.
2. Variant 2 Atomic: Increment the counter using a single atomic variable and memory order relaxed.
3. Variant 3 Sharded/unpadded: Increment the counter using an atomic variable *per* thread. The atomic variables are all forced onto the same cache line.
4. Variant 4 Sharded/Padded: Increment the counter using an atomic variable *per* thread. Each atomic is `alignas(CACHE_LINE_SIZE)`, 128 bytes on Mac and 64 on Linux, so every thread owns its own cache line.

Results

Timed Variants:

Variant	Mac BEST (ns/op)	Mac AVG (ns/op)	Linux BEST (ns/op)	Linux AVG (ns/op)
V1 mutex	12.746	13.531	48.782	~60-63 typical
V2 atomic (shared)	5.434	5.986	13.588	~15-16 typical
V3 sharded unpadded	5.171	5.483	13.265	~13-17 typical
V4 sharded padded	0.488	0.492	0.947	~0.95-1.00 typical

Linux Perf:

Execution metrics:

Variant	Cycles	Instructions	IPC	Time (s)	Inferred Freq
V1 mutex	103,027,010,844	67,933,474,958	0.659	7.512	-
V2 atomic (shared)	65,706,589,314	621,150,701	0.009	3.622	-
V3 sharded unpadded	67,600,825,816	623,506,785	0.009	3.679	-
V4 sharded padded	3,634,514,575	518,258,153	0.143	0.203	~4.47 GHz

Cache metrics:

Variant	LLC-load-misses	LLC-loads
V1 mutex	7,684	219,050,476
V2 atomic (shared)	6,264	262,796
V3 sharded unpadded	5,449	253,610
V4 sharded padded	4,218	120,372

- V1, V2, and V3 do not have a calculated frequency. V1 is because of futex overhead, V2/V3 are spin-stall bound, so their ratios do not cleanly reflect core frequency. Similarly, each test **ran 20 times**
- We still have some timing instructions - as such there are extra instructions included in the total instruction count. (Vector pushback)
- Instruction count is inflated, due to each test running 20 times.

Mac XCTrace

Execution metrics:

Variant	Cycles	INST_ALL	IPC
V1 mutex	2,662,199,566	240,216,469	0.090
V2 atomic (shared)	16,821,540,289	583,115,662	0.035
V3 sharded unpadding	14,939,848,481	561,025,495	0.038
V4 sharded padding	993,203,519	425,520,102	0.428

Cache metrics:

Variant	L1D Miss (LD)	L1D Miss (ST)	L1D TLB Miss	L2 TLB Miss
V1 mutex	228,824,687	296,724,007	10,695	1,220
V2 atomic (shared)	64,983,549	410,433,503	26,180	1,350
V3 sharded unpadding	59,769,250	391,299,663	37,406	1,854
V4 sharded padding	40,502	12,857	16,509	1,824

 Note

- Instruction count is inflated, due to each test running 20 times.

Findings:

Step-by-step speedup (best ns/op):

Transition	Mac speedup	Linux speedup
V1 → V2	2.35x	3.59x
V2 → V3	1.05x	1.02x
V3 → V4	10.6x	14.0x
V1 → V4	26.1x	51.5x

Speedup relative to V1 (best ns/op):

Variant	Mac (vs V1)	Linux (vs V1)
V1 mutex	1.0x	1.0x
V2 atomic (shared)	2.35x	3.59x
V3 sharded unpadded	2.47x	3.67x
V4 sharded padded	26.1x	51.5x

Cycle Difference relative to V1:

Variant	Mac (vs V1)	Linux (vs V1)
V1 mutex	1.0x (2.66B)	1.0x (103B)
V2 atomic (shared)	6.32x more	0.64x (36% fewer)
V3 sharded unpadded	5.61x more	0.66x (34% fewer)
V4 sharded padded	0.37x (2.68x fewer)	0.035x (28.3x fewer)

Cache difference relative to V1:

Variant	Mac CACHE_MISS_ST	Mac CACHE_MISS_LD	Linux LLC Miss
V1 mutex	1.0x (296.7M)	1.0x (228.8M)	1.0x (7,684)
V2 atomic (shared)	1.38x more	0.284x (71.6% fewer)	0.815x (18.5% fewer)
V3 sharded unpadded	1.32x more	0.261x (73.9% fewer)	0.709x (29.1% fewer)
V4 sharded padded	0.000043x (23,083x fewer)	0.000177x (5,649x fewer)	0.549x (45.1% fewer)

IPC difference relative to V1:

Variant	Mac (vs V1)	Linux (vs V1)
V1 mutex	1.0x (0.090)	1.0x (0.659)
V2 atomic (shared)	0.39x (0.035)	0.014x (0.009)
V3 sharded unpadded	0.42x (0.038)	0.014x (0.009)
V4 sharded padded	4.76x (0.428)	0.217x (0.143)

General Findings

- Mac V1
 - Has a better IPC, fewer cycles, and fewer instructions than V2, V3. This is due to the futex call. The threads are sleeping, and as such there is zero cycles wasted during the sleep time.
 - The kernel sleep time is huge, which explains the timing difference between V2, V3, V4.
- Mac V2/V3
 - Cycles are constantly spinning, as such their total cycles are extremely high.
- Linux V2/V3 Vs Mac V2/V3
 - Both platforms use a single hardware atomic instruction with no software retry loop. On x86, `fetch_add(relaxed)` compiles to `lock xadd`. On Mac ARM64, disassembly confirmed it compiles to `ldadd` (ARMv8.1 LSE), not an `ldxr/stxr` exclusive-monitor loop as initially assumed. Both instructions atomically read-modify-write the cache line in one hardware operation with no retry path.
 - The high cycle count for Mac V2/V3 comes from all four threads spinning at full speed on a contended `ldadd` with no stall hint in the counter increment loop. The cache line bounces between cores on every atomic, and there is no `yield` between attempts; threads burn cycles executing `ldadd` + loop overhead continuously. Linux threads do the same with `lock xadd`. `_mm_pause` reduces the Linux spin rate slightly, but the dominant factor is cache-line contention on both platforms.

Mac V1 VS Linux V1

Mac V1 is 12.746 ns/op whereas Linux V1 is 48.782 ns/op. That is, Linux is 3.82x slower. While we cannot determine the exact root cause due to Apple having most of this information secured as proprietary information, we can make educated inferences.

1. Different mutex implementations. glibc's `pthread_mutex` and Apple's (backed by the `unlock/os_unfair_lock` machinery) are different code with different spin-before-sleep policies. Some share of this gap is library, not hardware. The hardware inferences below sit on top of that.
2. Inter-core wakeup latency:
 1. On Linux, the 4 threads being used are pinned to isolated cores: 0, 1, 2, 3. These threads are physically separate cores connected by Intel's ring bus. That means an **IPI** (inter-processor interrupt) has to travel the ring bus to wake up thread B on another core. That round trip adds latency on every mutex handoff.
 2. On the M3 mac threads on the same P-core cluster share an L2 cache. That means the mutex state transfer and wakeup signal travel through shared cache, not a bus. The shared-L2 topology is verified via `sysctl`; that the mutex state transfer actually resolves there is inferred, not measured.
3. V1 Cycle Count
 1. Mac V1 cycles is 2.66 billion. Mac V2 cycles is 16.8 billion. V1 has **fewer** cycles than V2/V3. This low cycle count leads us to infer that the threads are sleeping, and mac wakes them up much faster than linux can.

V1 → V2 speedup

Let's start with our V1 → V2 speedup. Why does our lock take so long? That comes down to the specifics of how a locking mechanism works. A lock/unlock system works such as: `lock()`

1. Compare and Exchange Strong on the mutex state.
2. If CAS fails
 1. `futex` syscall
 2. thread sleeps
 3. kernel wakes it later
3. Full memory fence (Acquire semantics) `unlock()`
4. store mutex state back to 0.
5. Full memory fence (Release semantics)
6. If threads waiting -> `Futex` syscall to wake one

Atomicity on the other hand is much simpler.

- On x86: `lock xadd [mem], reg`. The hardware's cache coherency protocol handles atomicity.
- On ARM64: `ldadd` (ARMv8.1 LSE atomic add) confirmed via disassembly. Similarly hardware-level, single instruction, no OS involvement. So what are we actually

observing? Well atomicity is just much faster because:

1. Fewer instructions per call.
2. No OS involvement. Below are some steps that show why this matters:
 1. A **futex** call requires a privilege mode switch. As such we switch from user privilege to kernel privilege.
 2. The privilege switch means we have to save registers, run kernel code to manage wait queue, then return.
 3. The round trip can take anywhere from ~100-200ns on modern x86 before any sleeping happens.
3. No kernel involvement. Below are some steps that show why this matters:
 1. When a mutex is contended the thread calls **futex_wait**. As such the thread moves to a wait queue and is sleeping.
 2. Scheduler picks a new thread to run, so the old thread loses its cpu time slice and cache-warm state.
 3. **Futex_wake** at some point will be called on the old thread, (another syscall).

V2 → V3 speedup

V2 → V3 is actually a test that is inherently incorrect. The idea is to force **false sharing**. One would think that using 4 different atomic variables would speed up as threads are not sharing the variables anymore, meaning no collisions. However, you can see based on the struct we use that the 4 atomic variables are all defined in a contiguous array such that they are all on the same cache line. This is where **false sharing** comes from.

False sharing is the idea that we think we have speedup by having each thread own its own local variables. But it is not actual sharing, as the cache line will still be shared across multiple threads. This means each time a thread wants to read its local variable it needs to wait for the cache line to be broadcast on the cache interconnects, grab it, and update it, followed by putting it back out on the bus for the other caches.

As such, we do not see any speedup from V2 to V3. Each time the counter is incremented it dirties that cache line for the other threads, and they must re-retrieve the cache line.

V3 → V4 speedup

V3 → V4 has the greatest speedup. In V4 we eliminated the false sharing that was occurring. We did this using the **alignas** inline to force each atomic variable onto its own cache line. This allowed each thread to own a single cache line that its atomic

variable lived on. Meaning it never needed to share the cache line with the other threads. As such, there are no interrupts, no cache line misses, and no collisions. So what is actually happening? We have four threads operating with their own local variable. The only instructions truly needed are add and jump. This is because the threads have no contention, no shared cache lines, and no need to sleep. Similarly there are no wasted retry loops.

Hardware Counters Analysis (Linux x86, **perf stat**)

The IPC numbers expose the mechanism directly. V1 mutex IPC = 0.659: when a thread holds the lock it runs the increment with full cache-line ownership; the issue is all other threads are sleeping, costing wall time in serialization, not execution efficiency. V2 and V3 collapse to IPC 0.009: all four threads spin-stall waiting for the cache line, burning cycles without retiring instructions. V4 recovers to 0.143, 16x better than V2/V3, because the stalls are gone.

The LLC-load-misses are nearly identical across all variants (4,218-7,684) despite the 18x wall-time gap between V3 and V4. This is the key structural point: **false sharing is an L1/L2 phenomenon, not an LLC phenomenon**. In V2 and V3, the contended cache line bounces between per-core L1/L2 caches via the ring bus; the LLC (L3) acts as the coherence arbiter but the line stays cache-resident. Accesses that go to DRAM are rare in all variants because the entire working set fits in L3.

Hardware Counters Analysis (Mac ARM64, **Instruments**)

Mac tells a much different story than Linux. IPC numbers show V1 being much better than V2 and V3. However we already proved out this is due to V1 threads sleeping. So it appears it is faster, but when you look at the whole picture, including overall timing, we can dive into the truth.

Mac's IPC for V4 outdoes Linux by a lot. On Linux x86, **lock xadd** drains the store buffer and stalls the pipeline to assert cache line exclusivity. That increases the overhead even when there is no contention. On Mac ARM64 **ldadd** with memory order relaxed (confirmed via disassembly) means there is no barrier. The line is in M state in our L1 cache and no other core is contending, so **ldadd** completes with minimal overhead. This difference is an ISA-level cost. Mac V4 cache misses disappear. We see **L1D_CACHE_MISS_LD** go from 228M(V1) to 40K(V4), or a 5,649x reduction. Similarly, **L1D_CACHE_MISS_ST** goes from 296M(V1) to 12K(V4), a 23,083x reduction. This is due to the cache line staying in the "Modified" state with no other cache invalidating it. **CACHE_MISS_ST** jumps in V2/V3. Mac V1 store misses was 296 million, whereas V2

was 410 million, and V3 391 million (+38%). On Linux, removing the mutex *reduced* LLC activity. On Mac it increases store misses because each `ldadd` requires exclusive ownership of the cache line. When four threads contend on the same line, each successful `ldadd` invalidates the line in the other cores' caches; they must re-acquire it on their next attempt, generating a store miss. This coherency invalidation pattern is the Mac-specific mechanism behind the elevated `CACHE_MISS_ST`.

PingPong Benchmark

The PingPong benchmark is measuring an acquire/release round trip cost. That is, we have one thread attempting to acquire a variable and the other incrementing & releasing the variable. The p50 is the hardware cost of the primitive. The tail is environment: scheduler jitter on Mac, residual noise on isolated cores on Linux. For latency-sensitive systems the tail is the number that matters. The test has two different modes:

- 1. Mode 1: The flag and counter are on the same cache line.
- 2. Mode 2: The flag and counter are on separate cache lines.

Timed Results: Mode 1 (flag and counter on same cache line)

Percentile	Mac (ns)	Linux (cycles)	Linux (~ns, 2.808 GHz)
p0.1	41 ns	353	~126 ns
p1	41 ns	355	~126 ns
p25	83 ns	357	~127 ns
p50	83 ns	361	~129 ns
p75	84 ns	365	~130 ns
p99	125 ns	382	~136 ns
p99.9	250 ns	454	~162 ns

 **Mac values are tick-quantized to ~41.7 ns per tick. p0.1-p1 (1-tick = 41 ns), p25-p75 (2-tick = 83-84 ns), p99 (3-tick = 125 ns), p99.9 (6-tick = 250 ns). p50 and p99 are stable across runs; p99.9 reflects scheduler jitter (no hard affinity on macOS).**

Timed Results: Mode 2 (flag and counter on separate cache lines)

Percentile	Mac (ns)	Linux (cycles)	Linux (~ns, 2.808 GHz)
p0.1	83 ns	240	~86 ns
p1	83 ns	240	~86 ns
p25	83 ns	258	~92 ns
p50	84 ns	329	~117 ns
p75	125 ns	331	~118 ns
p99	167 ns	450	~160 ns
p99.9	250 ns	525	~187 ns

Hardware Counters (Linux x86, `perf stat`)

Mode	Cycles	Instructions	IPC	LLC-load-misses	LLC-loads
Mode 1 (same line)	13.6B	704,558,492	0.052	20,135	20.1M
Mode 2 (separate lines)	13.3B	692,502,880	0.052	16,752	20.3M

LLC-loads $\approx$ 20M in both modes (10M round trips x ~2 LLC-level loads per trip). LLC-load-misses are tiny (< 0.1% of LLC-loads), cross-core cache-line handoffs on the i9-10900 are resolved at the shared L3, not by going to DRAM. The IPC of 0.052 in both modes reflects the structure of a spin-wait: most cycles are spent in `while (flag.load() != expected) spin_hint()`, which retires very few instructions per cycle.

The two modes show nearly identical counters. On x86, the difference between a packed vs padded layout is whether the receiver's read also drags in the counter's cache line. In Mode 1 the flag and counter share a line, so the bounce carries both. In Mode 2 only the flag line bounces; the counter line stays with the sender. The ~1.9% cycle difference (13.6B vs 13.3B) is consistent with avoiding one extra cache-line transfer per trip.

Hardware Counters (Mac ARM64, `Instruments`)

Execution metrics:

Mode	Cycles	INST_ALL	IPC
Mode 1 (same line)	7,247,762,853	12,271,936,929	1.693
Mode 2 (separate lines)	7,851,653,567	13,262,640,769	1.689

Cache metrics:

Mode	L1D Miss (LD)	L1D Miss (ST)	L1D TLB Miss	L2 TLB Miss
Mode 1 (same line)	674,860,740	767,298	47,519	18,450
Mode 2 (separate lines)	622,108,964	1,319,796	785,504	65,741

Mac tells a much different story than Linux. Mode 1 and Mode 2 both sit at an IPC of 1.69, a 32x difference from Linux's 0.052. The ARM64 spin-wait (**ldapr** polling + branch) feeds M3's wide out-of-order back-end efficiently: the processor keeps the poll-check-branch loop flowing with no forced stall, retiring roughly one loop iteration every two cycles and sustaining IPC above 1.0. This is not a performance advantage. It means ARM's **yield** hint carries no guaranteed pipeline stall, so the OOO machine keeps issuing spin-loop iterations at full rate. Linux's 0.052 IPC reflects **_mm_pause** explicitly stalling the front-end to back off from the cache bus during contention.

Mode 2 runs 8.3% slower on Mac (7.85B vs 7.25B cycles), the opposite of Linux where Mode 2 was ~1.9% faster. The counters point at translation rather than coherency: **L1D_TLB_MISS** jumps 16.5x (47,519 to 785,504) and **L2_TLB_MISS_DATA** 3.6x (18,450 to 65,741) alongside the slowdown. A clean TLB story is hard to construct, though. macOS on Apple Silicon uses 16 KiB pages, and Mode 2's working set is a single 256-byte struct (**alignas(128)** on two 128-byte fields) that either straddles a page boundary once at allocation or never does. A static two-page footprint should not sustain ~785K misses across 10M round trips. The correlation is measured; the mechanism is unresolved. Next step: print the struct's address to check page placement, and re-run with the allocation forced within one page.

At the median (p50), Mac completes a round trip in 83 ns and Linux in ~129 ns (361 cycles at 2.808 GHz TSC). Mac is faster despite ARM64's explicit acquire/release instructions. The likely explanation is cluster topology: both pingpong threads are scheduled on the same P-core cluster, so the cache-line handoff resolves at the per-cluster shared L2 without touching the ring bus. On Intel, a cross-core handoff always

traverses the ring bus to reach the shared L3. This is inferred from cluster topology, not from published Apple microarchitecture specs.

At the tail (p99.9), Mac reaches 250 ns (6 ticks) against Linux's ~162 ns (454 cycles). The gap reflects different limiting phenomena. Linux runs both threads under `isolcpus + nohz_full + rcu_nocbs`; timer interrupts and RCU callbacks are eliminated from the benchmark cores, and the residual tail is hardware noise on isolated cores. Mac has no equivalent: `pthread_set_qos_class_self_np` is a scheduling hint only, and the kernel can preempt either thread for unrelated work. Mac's p99.9 is OS scheduler jitter; Linux's is hardware noise on isolated cores. They are not measuring the same phenomenon.

SPSC Queue Benchmark

A 1,024-slot lock-free SPSC ring buffer comparing two item layouts: `Item` (4 bytes, multiple slots per cache line) vs `PaddedItem` (`alignas(CACHE_LINE_SIZE)`, one item per cache line). Producer pinned to core 0, consumer to core 1, best-of-10 harness. `CACHE_LINE_SIZE` is 128 on Mac, 64 on Linux, so `PaddedItem` slot sizes differ between platforms.

Results

	Mac, Item	Mac, PaddedItem	Linux, Item	Linux, PaddedItem
Throughput (ops/sec)	30,400,776	34,190,419	41,821,745	46,286,482
p50 (ns)	42	41	14	15
p99 (ns)	42	42	67	47
p99.9 (ns)	84	84	71	50

✎ Mac values are tick-quantized to ~41.7 ns per tick. `PaddedItem` queue: 1,024 x 128 bytes = 128 KiB on Mac; 1,024 x 64 bytes = 64 KiB on Linux.

`PaddedItem` wins throughput on both platforms: +12.5% on Mac (34.2M vs 30.4M ops/s) and +10.7% on Linux (46.3M vs 41.8M ops/s). The mechanism is the same on both: temporal false sharing.

Hardware Counters (Linux x86, **perf stat**)

Mode	Cycles	Instructions	IPC	LLC-load-misses	LLC-loads
Item (shared cacheline)	2.7B	1,415,220,603	0.517	30,316	9.5M
PaddedItem (own cacheline)	3.1B	1,437,785,034	0.470	35,759	10.7M

The raw perf cycles (2.7B vs 3.1B) show **Item** using ~13% fewer total cycles, while the **_printed** timing run, a best-of-10 direct throughput measurement, shows **PaddedItem** ahead by 10.7%. The two measurements aggregate differently: perf sums all 10 internal iterations, cold and warm, across both cores, while **_printed** reports the best sustained run. Both facts stand as measured: **PaddedItem** wins best-case sustained throughput, **Item** wins aggregate cycles. Best sustained throughput is the question this benchmark was built to answer; the aggregate-cycle gap is noted rather than resolved.

The classic false-sharing framing does not directly apply here: the producer writes slot N, then N+1, then N+2 sequentially; the consumer reads them in the same order. No two threads access the same slot simultaneously. But in a fast, well-matched SPSC where the queue stays shallow, the producer and consumer operate within a few cache lines of each other. With **Item** (16 slots per 64-byte cache line on Linux), the producer writes to the same cache line the consumer just read, invalidating its L1 copy. The consumer re-fetches the line on the next access. This is **temporal false sharing**: overlapping cache line access in a short time window, not simultaneous access. **PaddedItem** breaks this by giving producer and consumer writes separate cache lines. The tail latency improvement (p99: 47 ns vs 67 ns) reflects fewer stall cycles waiting for L1 refills.

Hardware Counters (Mac ARM64, **Instruments**)

Execution metrics:

Mode	Cycles	INST_ALL	IPC
Item (shared cacheline)	2,374,475,888	3,603,057,162	1.517
PaddedItem (own cacheline)	2,398,923,278	2,912,323,923	1.214

Cache metrics:

Mode	L1D Miss (LD)	L1D Miss (ST)	L1D TLB Miss	L2 TLB Miss
Item (shared cacheline)	266,944,790	3,277,676	566,980	12,283
PaddedItem (own cacheline)	75,144,578	5,740,164	802,912	17,275

Mac's xctrace data makes the mechanism visible. `L1D_CACHE_MISS_LD` drops from 266.9M (`Item`) to 75.1M (`PaddedItem`), a 3.55x reduction, despite `Item`'s queue being 32x smaller (4 KB vs 128 KiB). The misses are not from working-set overflow; they come from producer writes invalidating the consumer's L1 copies. Padding each slot to its own 128-byte cache line eliminates the conflict.

Another reason Mac does much better than Linux is due to our shared L2 Cache. The Performance Core group shares an L2 Cache. Linux has to bump to the L3 cache for data to be shared; Mac only needs to go to L2.

`PaddedItem`'s IPC is lower (1.214 vs 1.517). The 128 KiB queue fills the M3 Pro P-core L1d (128 KiB) to capacity, leaving less room for control-path data. The throughput advantage (+12.5%) holds because the eliminated cross-core invalidations outweigh the L1 capacity pressure, at this queue depth. At a larger slot count the 128 KiB padded footprint would spill further past L1 and the trade could flip; not measured here.

What the Hardware Docs Say

Intel. The SDM Vol. 3A §8 defines x86-TSO: loads are not reordered with other loads, stores are not reordered with other stores, and the one reordering the architecture permits is a store passing a later load (StoreLoad). That contract is why `memory_order_acquire/release` compiled to plain `mov` in every hot path here, and why only `seq_cst` pays for an explicit fence or locked operation on x86. The Optimization Reference Manual covers `pause`, the spin-loop de-pipelining that shows up in this data as Linux's 0.052 IPC.

ARM. The Architecture Reference Manual defines a weakly ordered baseline with three explicit barriers: `DMB` (ordering), `DSB` (completion), and `ISB` (context synchronization). None of them appear in these binaries. Disassembly showed all ordering carried by the one-sided instructions themselves, `ldapr` (RCpc load-acquire, ARMv8.3) and `stlr`

(store-release), which is the mapping compilers use for C++ acquire/release on ARMv8.3+ hardware.

Apple. There is no published microarchitecture manual for Apple Silicon. No cache topology document, no coherence protocol description, no optimization guide. Every Apple-internals claim in this post is therefore one of three things, labeled where it appears: queried directly (`sysctl` line size and cache sizes), measured (timing, PMU counters, disassembly), or inferred. For inference the best available public source is Dougall Johnson's reverse-engineering of Apple cores, which is community work, not a spec, and cited as such.

Takeaways for Systems Engineers

- **Results don't transfer across ISAs, and sometimes they invert.** The same C++ produced a 26.1x V1 to V4 gain on the M3 Pro and 51.5x on the i9-10900, and the Mode 2 padding change was ~1.9% faster on Linux but 8.3% slower on Mac. Benchmark on the hardware you'll deploy on.
- **Verify the instructions you think you're benchmarking.** I assumed `fetch_add` compiled to an `ldxr/stxr` retry loop. Disassembly showed a single `ldadd`, and that correction rewrote three separate mechanism explanations. `objdump` costs one command.
- **Know what your spin hint actually does.** `_mm_pause` stalled the x86 front-end to 0.052 IPC. ARM's `yield` left the M3's out-of-order engine running the poll loop at 1.69 IPC. Same source code, opposite pipeline behavior.
- **Core placement control shows up in the tail, not the median.** On isolated Linux cores (`isolcpus` + `nohz_full` + `rcu_nocbs`), pingpong p99.9 is 454 cycles, 1.26x the median. On Mac, with no hard affinity API, p99.9 runs 3x the median (250 ns vs 83 ns, tick-quantized). Same primitive, different tail discipline.

ARM64 vs x86 Synchronization: What the Numbers Actually Say

Introduction

Recently I rotated through an engineering team at Caterpillar Inc. where I developed low-level software for Telematics Engine Control Units on processors running Linux. That was a step outside my normal path, I had focused on low-level software for RTOS machines for a majority of my time at Cat.

Moving into lower-latency systems work raised a question I couldn't answer from documentation alone: how much does ISA-level **memory** ordering actually cost, and does the ARM64/x86 gap matter in practice?

On ARMv8 the compiler must emit ordering-bearing instructions for `memory_order_acquire` and `memory_order_release`. On my hardware those are `ldapr` and `stlr`, confirmed by disassembly below, with semantics defined in the ARM Architecture Reference Manual. Intel's SDM Vol. 3A §8 describes x86's TSO model: the hardware already enforces those ordering constraints, so acquire/release compile to plain loads and stores. Different contracts. I wanted the measured delta, not the architectural description.

I ran three benchmarks on two physical machines: a MacBook Pro M3 Pro (ARM64, macOS) and a Dell Precision 3640 running Ubuntu bare metal (Intel 10th Gen Comet Lake, x86-64). Bare metal matters, a hypervisor remaps vCPUs between physical cores and pollutes both pinning assumptions and tail latency.

What follows are the results from a counter contention ladder (V1-V4), a pingpong acquire/release round-trip, and an SPSC queue throughput test, with the mechanism behind each number.

Hardware Specifications

	MacBook Pro (ARM64)	Dell Precision 3640 (x86-64)
CPU	Apple M3 Pro (6P + 6E cores, 12 total)	Intel i9-10900, 10C/20T, Comet Lake
Architecture	ARM64	x86-64
OS	macOS Sequoia 15.7.3	Ubuntu 24.04 (bare metal)

	MacBook Pro (ARM64)	Dell Precision 3640 (x86-64)
RAM	36 GB	32 GB
Kernel	24.6.0	6.8.0-28-generic
Cache Line	128 bytes (measured via <code>sysctl</code>)	64 bytes
L1d / L1i	P-core: 128 KiB / 192 KiB; E-core: 64 KiB / 128 KiB (measured; no published spec)	32 KiB / 32 KiB per core
L2	P-core: 16 MiB per cluster; E-core: 4 MiB (measured; no published spec)	256 KiB per core
L3	No conventional L3; a system-level cache serves the role (size unpublished; inferred)	20 MiB (shared)
Timer	<code>mach_absolute_time</code> , 24 MHz, ~42 ns/tick	<code>rdtscp</code> , serialized cycle counter
Thread Pinning	<code>pthread_set_qos_class_self_np</code> (QoS hint only, no hard affinity API)	<code>pthread_setaffinity_np</code>
Core Isolation	N/A	<code>isolcpus=0,1,2,3</code> + <code>nohz_full</code> + <code>rcu_nocbs</code>

How I Measured

Timing

Both platforms use the highest-resolution timer available without OS involvement on the hot path.

Linux (x86-64): `rdtscp`, Intel's serializing timestamp counter. Serializing here means `rdtscp` waits for all prior instructions to complete before reading the counter. It does not stop later instructions from starting early, so it bounds the start of a measured region tightly, and at these interval lengths the end is bounded well enough that no trailing fence was added. The TSC is invariant on the i9-10900: it ticks at a fixed frequency regardless of CPU boost state or C-state transitions. That frequency was not assumed from the spec sheet; it was calibrated at runtime by correlating `rdtscp` deltas against

`CLOCK_MONOTONIC_RAW` over a 200 ms `nanosleep` interval. Measured TSC frequency: **2.808 GHz**. All cycle counts in this document convert to nanoseconds using that calibrated figure.

macOS (ARM64): `mach_absolute_time()` converted via `mach_timebase_info`, Apple's recommended high-resolution timer. On M3 Pro this ticks at 24 MHz (~41.7 ns per tick). This is a hard resolution floor: any latency value reported on Mac is quantized to the nearest tick boundary. Throughput measurements are reliable; latency percentiles are tick counts, not exact nanoseconds. This quantization is called out explicitly wherever it affects interpretation.

CPU Frequency and Stability

Linux: CPU governor was set to `performance` via `cpupower frequency-set -g performance`, and `intel_pstate` minimum performance was pinned to 100% (`/sys/devices/system/cpu/intel_pstate/min_perf_pct = 100`) to prevent idle clock-down between benchmark phases. Actual operating frequency was verified via `perf stat` cycles divided by wall time; `scaling_cur_freq` on this `intel_pstate` system reflects idle P-state snapshots, not the turbo frequency achieved during the workload. Turbo boost confirmed enabled (`no_turbo = 0`).

macOS: No equivalent frequency-pinning API exists.

`pthread_set_qos_class_self_np(QOS_CLASS_USER_INTERACTIVE, 0)` was used as a scheduling hint. Low Power Mode was disabled for all runs. On Apple Silicon this mode caps P-core frequency and produces 2-3x slowdowns that are indistinguishable from code regressions without an explicit A/B check. `macmon` was used to monitor cluster frequencies and confirm the M3 Pro was not throttling during benchmark runs.

Thread Pinning and Core Isolation

Linux: `pthread_setaffinity_np` pins each thread to a specific physical core. Cores 0-3 were removed from the kernel scheduler entirely via kernel boot parameters (`isolcpus=0,1,2,3 nohz_full=0,1,2,3 rcu_nocbs=0,1,2,3`). This eliminates timer interrupts and RCU callbacks on the benchmark cores. The pingpong tails were collected under this configuration, and the Mac/Linux tail comparison in that section reflects the asymmetry.

macOS: Hard affinity is not available; `pthread_setaffinity_np` does not exist on macOS. The QoS hint used here was tested and did not measurably tighten variance on contended variants. The macOS results therefore carry higher tail sensitivity to OS scheduler noise than the Linux results. This asymmetry is the primary reason Mac and Linux p99.9 values are not directly comparable.

Hardware Counter Collection

Linux: `perf stat` wraps the benchmark binary to collect aggregate hardware counters (LLC load misses, instructions, cycles) without modifying the binary. Counter collection and timing runs are kept separate, so the `perf` overhead does not appear in the latency numbers reported in the results tables.

```
perf stat -e cycles,instructions,LLC-load-misses,LLC-loads ./pingpong_bench
```

macOS: `xctrace` with the "CPU Counters" template is the equivalent; it accesses Apple Silicon PMU counters from the terminal without requiring the Xcode GUI. Like `perf stat`, it is run separately from the timing benchmark to avoid overhead contaminating the latency results.

```
xctrace record --template "HW_Counters" --launch -- ./pingpong_bench --  
output counters.trace
```

The resulting `.trace` file is inspected in Instruments. Note that Apple Silicon PMU counter names do not map 1:1 to Intel's (e.g., there is no direct `LLC-load-misses` equivalent by that name). The specific counters used and their Apple Silicon labels are noted alongside each result where hardware counter data is cited.

Instruction Verification (macOS)

To verify that the compiler emitted the expected instructions on Apple Silicon and that no unintended barriers or retry loops appeared in the hot paths, all three binaries were disassembled using:

```
xcrun llvm-objdump -d --mcpu=apple-m3 ./build/bench/<binary> >
notes/external/disasm_mac.txt
```

The resulting output was grepped for the instructions that the paper's analysis depends on. Key findings:

ldadd: ARMv8.1 LSE atomic add. `counter_bench`'s `fetch_add(memory_order_relaxed)` compiles to a single `ldadd` instruction, not an `ldxr/stxr` exclusive-monitor loop. `ldadd` atomically reads a memory location, adds a value, and writes the result back in one hardware operation. Like x86's `lock xadd`, it never fails and has no software retry path. M3 Pro supports ARMv8.1 Large System Extensions, so the compiler emits this rather than the older exclusive-monitor pattern.

ldapr: Load-Acquire RCpc. `pingpong_bench`'s `memory_order_acquire` load emits `ldapr`, not the older `ldar`. RCPC (Release Consistency, Processor Consistent) provides acquire semantics against prior stores without the full cost of `ldar`'s TSO-equivalent barrier. This is the ARMv8.3 RCPC extension, available on Apple Silicon.

stlr: Store-Release. `memory_order_release` stores emit `stlr` across all benchmarks. This is the base ARMv8 store-release instruction; it prevents the store from being observed before any prior memory accesses.

yield: Spin hint. The spin-wait loop in `pingpong_bench` emits the `yield` instruction (the ARM equivalent of x86 `_mm_pause`). It hints to the CPU that the core is in a spin-wait, allowing the hardware to reduce power and back off resource contention. Unlike `_mm_pause` on x86, `yield` carries no guaranteed pipeline stall duration; its effect is microarchitecture-dependent and measurably weaker.

No dmb/dsb/isb. No explicit memory barrier instructions appear in any hot path. All ordering is carried by the `ldapr/stlr` instructions themselves, consistent with the ARM ARM's specification for acquire/release on ARMv8.3+.

No ldxr/stxr. The exclusive-monitor loop does not appear anywhere in the counter or pingpong hot paths. This corrects an earlier assumption in the analysis; see General Findings below.

Sampling Strategy

All benchmarks use an in-process best-of-N harness rather than re-launching the binary N times. Relaunching introduces inter-run OS and thermal entropy; in-process sampling amortizes cold-start cost across one binary invocation. Counter uses a best AND average of 20, pingpong was re-ran 10 times, SPSC uses best-of-10 with 10 internal iterations. Raw samples are sorted and percentiles are computed directly, with no smoothing or outlier removal.

Testing

We have three benchmarks we are going to look at. Counter Benchmark, Pingpong Benchmark, and SPSC Benchmark. Below you will find details and test results for each benchmark.

Counter Benchmark (V1-V4)

Test Variations

The first series of tests is 4 variations of incrementing a counter 10 million times across 4 threads. Each variation removes a weakness the previous variation had. That is we start with a lock, then move to atomicity, then multiple atomic variables, followed by independent cachelines.

1. Variant 1 Mutex: Increment the counter using a basic lock/unlock.
2. Variant 2 Atomic: Increment the counter using a single atomic variable and memory order relaxed.
3. Variant 3 Sharded/unpadded: Increment the counter using an atomic variable *per* thread. The atomic variables are all forced onto the same cache line.
4. Variant 4 Sharded/Padded: Increment the counter using an atomic variable *per* thread. Each atomic is `alignas(CACHE_LINE_SIZE)`, 128 bytes on Mac and 64 on Linux, so every thread owns its own cache line.

Results

Timed Variants:

Variant	Mac BEST (ns/op)	Mac AVG (ns/op)	Linux BEST (ns/op)	Linux AVG (ns/op)
V1 mutex	12.746	13.531	48.782	~60-63 typical
V2 atomic (shared)	5.434	5.986	13.588	~15-16 typical
V3 sharded unpadded	5.171	5.483	13.265	~13-17 typical
V4 sharded padded	0.488	0.492	0.947	~0.95-1.00 typical

Linux Perf:

Execution metrics:

Variant	Cycles	Instructions	IPC	Time (s)	Inferred Freq
V1 mutex	103,027,010,844	67,933,474,958	0.659	7.512	-
V2 atomic (shared)	65,706,589,314	621,150,701	0.009	3.622	-
V3 sharded unpadded	67,600,825,816	623,506,785	0.009	3.679	-
V4 sharded padded	3,634,514,575	518,258,153	0.143	0.203	~4.47 GHz

Cache metrics:

Variant	LLC-load-misses	LLC-loads
V1 mutex	7,684	219,050,476
V2 atomic (shared)	6,264	262,796
V3 sharded unpadded	5,449	253,610
V4 sharded padded	4,218	120,372

- V1, V2, and V3 do not have a calculated frequency. V1 is because of futex overhead, V2/V3 are spin-stall bound, so their ratios do not cleanly reflect core frequency. Similarly, each test **ran 20 times**
- We still have some timing instructions - as such there are extra instructions included in the total instruction count. (Vector pushback)
- Instruction count is inflated, due to each test running 20 times.

Mac XCTrace

Execution metrics:

Variant	Cycles	INST_ALL	IPC
V1 mutex	2,662,199,566	240,216,469	0.090
V2 atomic (shared)	16,821,540,289	583,115,662	0.035
V3 sharded unpadding	14,939,848,481	561,025,495	0.038
V4 sharded padding	993,203,519	425,520,102	0.428

Cache metrics:

Variant	L1D Miss (LD)	L1D Miss (ST)	L1D TLB Miss	L2 TLB Miss
V1 mutex	228,824,687	296,724,007	10,695	1,220
V2 atomic (shared)	64,983,549	410,433,503	26,180	1,350
V3 sharded unpadding	59,769,250	391,299,663	37,406	1,854
V4 sharded padding	40,502	12,857	16,509	1,824

 Note

- Instruction count is inflated, due to each test running 20 times.

Findings:

Step-by-step speedup (best ns/op):

Transition	Mac speedup	Linux speedup
V1 → V2	2.35x	3.59x
V2 → V3	1.05x	1.02x
V3 → V4	10.6x	14.0x
V1 → V4	26.1x	51.5x

Speedup relative to V1 (best ns/op):

Variant	Mac (vs V1)	Linux (vs V1)
V1 mutex	1.0x	1.0x
V2 atomic (shared)	2.35x	3.59x
V3 sharded unpadded	2.47x	3.67x
V4 sharded padded	26.1x	51.5x

Cycle Difference relative to V1:

Variant	Mac (vs V1)	Linux (vs V1)
V1 mutex	1.0x (2.66B)	1.0x (103B)
V2 atomic (shared)	6.32x more	0.64x (36% fewer)
V3 sharded unpadded	5.61x more	0.66x (34% fewer)
V4 sharded padded	0.37x (2.68x fewer)	0.035x (28.3x fewer)

Cache difference relative to V1:

Variant	Mac CACHE_MISS_ST	Mac CACHE_MISS_LD	Linux LLC Miss
V1 mutex	1.0x (296.7M)	1.0x (228.8M)	1.0x (7,684)
V2 atomic (shared)	1.38x more	0.284x (71.6% fewer)	0.815x (18.5% fewer)
V3 sharded unpadded	1.32x more	0.261x (73.9% fewer)	0.709x (29.1% fewer)
V4 sharded padded	0.000043x (23,083x fewer)	0.000177x (5,649x fewer)	0.549x (45.1% fewer)

IPC difference relative to V1:

Variant	Mac (vs V1)	Linux (vs V1)
V1 mutex	1.0x (0.090)	1.0x (0.659)
V2 atomic (shared)	0.39x (0.035)	0.014x (0.009)
V3 sharded unpadded	0.42x (0.038)	0.014x (0.009)
V4 sharded padded	4.76x (0.428)	0.217x (0.143)

General Findings

- Mac V1
 - Has a better IPC, fewer cycles, and fewer instructions than V2, V3. This is due to the futex call. The threads are sleeping, and as such there is zero cycles wasted during the sleep time.
 - The kernel sleep time is huge, which explains the timing difference between V2, V3, V4.
- Mac V2/V3
 - Cycles are constantly spinning, as such their total cycles are extremely high.
- Linux V2/V3 Vs Mac V2/V3
 - Both platforms use a single hardware atomic instruction with no software retry loop. On x86, `fetch_add(relaxed)` compiles to `lock xadd`. On Mac ARM64, disassembly confirmed it compiles to `ldadd` (ARMv8.1 LSE), not an `ldxr/stxr` exclusive-monitor loop as initially assumed. Both instructions atomically read-modify-write the cache line in one hardware operation with no retry path.
 - The high cycle count for Mac V2/V3 comes from all four threads spinning at full speed on a contended `ldadd` with no stall hint in the counter increment loop. The cache line bounces between cores on every atomic, and there is no `yield` between attempts; threads burn cycles executing `ldadd` + loop overhead continuously. Linux threads do the same with `lock xadd`. `_mm_pause` reduces the Linux spin rate slightly, but the dominant factor is cache-line contention on both platforms.

Mac V1 VS Linux V1

Mac V1 is 12.746 ns/op whereas Linux V1 is 48.782 ns/op. That is, Linux is 3.82x slower. While we cannot determine the exact root cause due to Apple having most of this information secured as proprietary information, we can make educated inferences.

1. Different mutex implementations. glibc's `pthread_mutex` and Apple's (backed by the `unlock/os_unfair_lock` machinery) are different code with different spin-before-sleep policies. Some share of this gap is library, not hardware. The hardware inferences below sit on top of that.
2. Inter-core wakeup latency:
 1. On Linux, the 4 threads being used are pinned to isolated cores: 0, 1, 2, 3. These threads are physically separate cores connected by Intel's ring bus. That means an **IPI** (inter-processor interrupt) has to travel the ring bus to wake up thread B on another core. That round trip adds latency on every mutex handoff.
 2. On the M3 mac threads on the same P-core cluster share an L2 cache. That means the mutex state transfer and wakeup signal travel through shared cache, not a bus. The shared-L2 topology is verified via `sysctl`; that the mutex state transfer actually resolves there is inferred, not measured.
3. V1 Cycle Count
 1. Mac V1 cycles is 2.66 billion. Mac V2 cycles is 16.8 billion. V1 has **fewer** cycles than V2/V3. This low cycle count leads us to infer that the threads are sleeping, and mac wakes them up much faster than linux can.

V1 → V2 speedup

Let's start with our V1 → V2 speedup. Why does our lock take so long? That comes down to the specifics of how a locking mechanism works. A lock/unlock system works such as: `lock()`

1. Compare and Exchange Strong on the mutex state.
2. If CAS fails
 1. `futex` syscall
 2. thread sleeps
 3. kernel wakes it later
3. Full memory fence (Acquire semantics) `unlock()`
4. store mutex state back to 0.
5. Full memory fence (Release semantics)
6. If threads waiting -> `Futex` syscall to wake one

Atomicity on the other hand is much simpler.

- On x86: `lock xadd [mem], reg`. The hardware's cache coherency protocol handles atomicity.
- On ARM64: `ldadd` (ARMv8.1 LSE atomic add) confirmed via disassembly. Similarly hardware-level, single instruction, no OS involvement. So what are we actually

observing? Well atomicity is just much faster because:

1. Fewer instructions per call.
2. No OS involvement. Below are some steps that show why this matters:
 1. A **futex** call requires a privilege mode switch. As such we switch from user privilege to kernel privilege.
 2. The privilege switch means we have to save registers, run kernel code to manage wait queue, then return.
 3. The round trip can take anywhere from ~100-200ns on modern x86 before any sleeping happens.
3. No kernel involvement. Below are some steps that show why this matters:
 1. When a mutex is contended the thread calls **futex_wait**. As such the thread moves to a wait queue and is sleeping.
 2. Scheduler picks a new thread to run, so the old thread loses its cpu time slice and cache-warm state.
 3. **Futex_wake** at some point will be called on the old thread, (another syscall).

V2 → V3 speedup

V2 → V3 is actually a test that is inherently incorrect. The idea is to force **false sharing**. One would think that using 4 different atomic variables would speed up as threads are not sharing the variables anymore, meaning no collisions. However, you can see based on the struct we use that the 4 atomic variables are all defined in a contiguous array such that they are all on the same cache line. This is where **false sharing** comes from.

False sharing is the idea that we think we have speedup by having each thread own its own local variables. But it is not actual sharing, as the cache line will still be shared across multiple threads. This means each time a thread wants to read its local variable it needs to wait for the cache line to be broadcast on the cache interconnects, grab it, and update it, followed by putting it back out on the bus for the other caches.

As such, we do not see any speedup from V2 to V3. Each time the counter is incremented it dirties that cache line for the other threads, and they must re-retrieve the cache line.

V3 → V4 speedup

V3 → V4 has the greatest speedup. In V4 we eliminated the false sharing that was occurring. We did this using the **alignas** inline to force each atomic variable onto its own cache line. This allowed each thread to own a single cache line that its atomic

variable lived on. Meaning it never needed to share the cache line with the other threads. As such, there are no interrupts, no cache line misses, and no collisions. So what is actually happening? We have four threads operating with their own local variable. The only instructions truly needed are add and jump. This is because the threads have no contention, no shared cache lines, and no need to sleep. Similarly there are no wasted retry loops.

Hardware Counters Analysis (Linux x86, **perf stat**)

The IPC numbers expose the mechanism directly. V1 mutex IPC = 0.659: when a thread holds the lock it runs the increment with full cache-line ownership; the issue is all other threads are sleeping, costing wall time in serialization, not execution efficiency. V2 and V3 collapse to IPC 0.009: all four threads spin-stall waiting for the cache line, burning cycles without retiring instructions. V4 recovers to 0.143, 16x better than V2/V3, because the stalls are gone.

The LLC-load-misses are nearly identical across all variants (4,218-7,684) despite the 18x wall-time gap between V3 and V4. This is the key structural point: **false sharing is an L1/L2 phenomenon, not an LLC phenomenon**. In V2 and V3, the contended cache line bounces between per-core L1/L2 caches via the ring bus; the LLC (L3) acts as the coherence arbiter but the line stays cache-resident. Accesses that go to DRAM are rare in all variants because the entire working set fits in L3.

Hardware Counters Analysis (Mac ARM64, **Instruments**)

Mac tells a much different story than Linux. IPC numbers show V1 being much better than V2 and V3. However we already proved out this is due to V1 threads sleeping. So it appears it is faster, but when you look at the whole picture, including overall timing, we can dive into the truth.

Mac's IPC for V4 outdoes Linux by a lot. On Linux x86, **lock xadd** drains the store buffer and stalls the pipeline to assert cache line exclusivity. That increases the overhead even when there is no contention. On Mac ARM64 **ldadd** with memory order relaxed (confirmed via disassembly) means there is no barrier. The line is in M state in our L1 cache and no other core is contending, so **ldadd** completes with minimal overhead. This difference is an ISA-level cost. Mac V4 cache misses disappear. We see **L1D_CACHE_MISS_LD** go from 228M(V1) to 40K(V4), or a 5,649x reduction. Similarly, **L1D_CACHE_MISS_ST** goes from 296M(V1) to 12K(V4), a 23,083x reduction. This is due to the cache line staying in the "Modified" state with no other cache invalidating it. **CACHE_MISS_ST** jumps in V2/V3. Mac V1 store misses was 296 million, whereas V2

was 410 million, and V3 391 million (+38%). On Linux, removing the mutex *reduced* LLC activity. On Mac it increases store misses because each `ldadd` requires exclusive ownership of the cache line. When four threads contend on the same line, each successful `ldadd` invalidates the line in the other cores' caches; they must re-acquire it on their next attempt, generating a store miss. This coherency invalidation pattern is the Mac-specific mechanism behind the elevated `CACHE_MISS_ST`.

PingPong Benchmark

The PingPong benchmark is measuring an acquire/release round trip cost. That is, we have one thread attempting to acquire a variable and the other incrementing & releasing the variable. The p50 is the hardware cost of the primitive. The tail is environment: scheduler jitter on Mac, residual noise on isolated cores on Linux. For latency-sensitive systems the tail is the number that matters. The test has two different modes:

- 1. Mode 1: The flag and counter are on the same cache line.
- 2. Mode 2: The flag and counter are on separate cache lines.

Timed Results: Mode 1 (flag and counter on same cache line)

Percentile	Mac (ns)	Linux (cycles)	Linux (~ns, 2.808 GHz)
p0.1	41 ns	353	~126 ns
p1	41 ns	355	~126 ns
p25	83 ns	357	~127 ns
p50	83 ns	361	~129 ns
p75	84 ns	365	~130 ns
p99	125 ns	382	~136 ns
p99.9	250 ns	454	~162 ns

 **Mac values are tick-quantized to ~41.7 ns per tick. p0.1-p1 (1-tick = 41 ns), p25-p75 (2-tick = 83-84 ns), p99 (3-tick = 125 ns), p99.9 (6-tick = 250 ns). p50 and p99 are stable across runs; p99.9 reflects scheduler jitter (no hard affinity on macOS).**

Timed Results: Mode 2 (flag and counter on separate cache lines)

Percentile	Mac (ns)	Linux (cycles)	Linux (~ns, 2.808 GHz)
p0.1	83 ns	240	~86 ns
p1	83 ns	240	~86 ns
p25	83 ns	258	~92 ns
p50	84 ns	329	~117 ns
p75	125 ns	331	~118 ns
p99	167 ns	450	~160 ns
p99.9	250 ns	525	~187 ns

Hardware Counters (Linux x86, **perf stat**)

Mode	Cycles	Instructions	IPC	LLC-load-misses	LLC-loads
Mode 1 (same line)	13.6B	704,558,492	0.052	20,135	20.1M
Mode 2 (separate lines)	13.3B	692,502,880	0.052	16,752	20.3M

LLC-loads $\approx$ 20M in both modes (10M round trips x ~2 LLC-level loads per trip). LLC-load-misses are tiny ($< 0.1\%$ of LLC-loads), cross-core cache-line handoffs on the i9-10900 are resolved at the shared L3, not by going to DRAM. The IPC of 0.052 in both modes reflects the structure of a spin-wait: most cycles are spent in **while (flag.load() != expected) spin_hint()**, which retires very few instructions per cycle.

The two modes show nearly identical counters. On x86, the difference between a packed vs padded layout is whether the receiver's read also drags in the counter's cache line. In Mode 1 the flag and counter share a line, so the bounce carries both. In Mode 2 only the flag line bounces; the counter line stays with the sender. The ~1.9% cycle difference (13.6B vs 13.3B) is consistent with avoiding one extra cache-line transfer per trip.

Hardware Counters (Mac ARM64, **Instruments**)

Execution metrics:

Mode	Cycles	INST_ALL	IPC
Mode 1 (same line)	7,247,762,853	12,271,936,929	1.693
Mode 2 (separate lines)	7,851,653,567	13,262,640,769	1.689

Cache metrics:

Mode	L1D Miss (LD)	L1D Miss (ST)	L1D TLB Miss	L2 TLB Miss
Mode 1 (same line)	674,860,740	767,298	47,519	18,450
Mode 2 (separate lines)	622,108,964	1,319,796	785,504	65,741

Mac tells a much different story than Linux. Mode 1 and Mode 2 both sit at an IPC of 1.69, a 32x difference from Linux's 0.052. The ARM64 spin-wait (**ldapr** polling + branch) feeds M3's wide out-of-order back-end efficiently: the processor keeps the poll-check-branch loop flowing with no forced stall, retiring roughly one loop iteration every two cycles and sustaining IPC above 1.0. This is not a performance advantage. It means ARM's **yield** hint carries no guaranteed pipeline stall, so the OOO machine keeps issuing spin-loop iterations at full rate. Linux's 0.052 IPC reflects **_mm_pause** explicitly stalling the front-end to back off from the cache bus during contention.

Mode 2 runs 8.3% slower on Mac (7.85B vs 7.25B cycles), the opposite of Linux where Mode 2 was ~1.9% faster. The counters point at translation rather than coherency: **L1D_TLB_MISS** jumps 16.5x (47,519 to 785,504) and **L2_TLB_MISS_DATA** 3.6x (18,450 to 65,741) alongside the slowdown. A clean TLB story is hard to construct, though. macOS on Apple Silicon uses 16 KiB pages, and Mode 2's working set is a single 256-byte struct (**alignas(128)** on two 128-byte fields) that either straddles a page boundary once at allocation or never does. A static two-page footprint should not sustain ~785K misses across 10M round trips. The correlation is measured; the mechanism is unresolved. Next step: print the struct's address to check page placement, and re-run with the allocation forced within one page.

At the median (p50), Mac completes a round trip in 83 ns and Linux in ~129 ns (361 cycles at 2.808 GHz TSC). Mac is faster despite ARM64's explicit acquire/release instructions. The likely explanation is cluster topology: both pingpong threads are scheduled on the same P-core cluster, so the cache-line handoff resolves at the per-cluster shared L2 without touching the ring bus. On Intel, a cross-core handoff always

traverses the ring bus to reach the shared L3. This is inferred from cluster topology, not from published Apple microarchitecture specs.

At the tail (p99.9), Mac reaches 250 ns (6 ticks) against Linux's ~162 ns (454 cycles). The gap reflects different limiting phenomena. Linux runs both threads under `isolcpus + nohz_full + rcu_nocbs`; timer interrupts and RCU callbacks are eliminated from the benchmark cores, and the residual tail is hardware noise on isolated cores. Mac has no equivalent: `pthread_set_qos_class_self_np` is a scheduling hint only, and the kernel can preempt either thread for unrelated work. Mac's p99.9 is OS scheduler jitter; Linux's is hardware noise on isolated cores. They are not measuring the same phenomenon.

SPSC Queue Benchmark

A 1,024-slot lock-free SPSC ring buffer comparing two item layouts: `Item` (4 bytes, multiple slots per cache line) vs `PaddedItem` (`alignas(CACHE_LINE_SIZE)`, one item per cache line). Producer pinned to core 0, consumer to core 1, best-of-10 harness. `CACHE_LINE_SIZE` is 128 on Mac, 64 on Linux, so `PaddedItem` slot sizes differ between platforms.

Results

	Mac, Item	Mac, PaddedItem	Linux, Item	Linux, PaddedItem
Throughput (ops/sec)	30,400,776	34,190,419	41,821,745	46,286,482
p50 (ns)	42	41	14	15
p99 (ns)	42	42	67	47
p99.9 (ns)	84	84	71	50

✎ Mac values are tick-quantized to ~41.7 ns per tick. `PaddedItem` queue: 1,024 x 128 bytes = 128 KiB on Mac; 1,024 x 64 bytes = 64 KiB on Linux.

`PaddedItem` wins throughput on both platforms: +12.5% on Mac (34.2M vs 30.4M ops/s) and +10.7% on Linux (46.3M vs 41.8M ops/s). The mechanism is the same on both: temporal false sharing.

Hardware Counters (Linux x86, perf stat)

Mode	Cycles	Instructions	IPC	LLC-load-misses	LLC-loads
Item (shared cacheline)	2.7B	1,415,220,603	0.517	30,316	9.5M
PaddedItem (own cacheline)	3.1B	1,437,785,034	0.470	35,759	10.7M

The raw perf cycles (2.7B vs 3.1B) show **Item** using ~13% fewer total cycles, while the **_printed** timing run, a best-of-10 direct throughput measurement, shows **PaddedItem** ahead by 10.7%. The two measurements aggregate differently: perf sums all 10 internal iterations, cold and warm, across both cores, while **_printed** reports the best sustained run. Both facts stand as measured: **PaddedItem** wins best-case sustained throughput, **Item** wins aggregate cycles. Best sustained throughput is the question this benchmark was built to answer; the aggregate-cycle gap is noted rather than resolved.

The classic false-sharing framing does not directly apply here: the producer writes slot N, then N+1, then N+2 sequentially; the consumer reads them in the same order. No two threads access the same slot simultaneously. But in a fast, well-matched SPSC where the queue stays shallow, the producer and consumer operate within a few cache lines of each other. With **Item** (16 slots per 64-byte cache line on Linux), the producer writes to the same cache line the consumer just read, invalidating its L1 copy. The consumer re-fetches the line on the next access. This is **temporal false sharing**: overlapping cache line access in a short time window, not simultaneous access. **PaddedItem** breaks this by giving producer and consumer writes separate cache lines. The tail latency improvement (p99: 47 ns vs 67 ns) reflects fewer stall cycles waiting for L1 refills.

Hardware Counters (Mac ARM64, Instruments)

Execution metrics:

Mode	Cycles	INST_ALL	IPC
Item (shared cacheline)	2,374,475,888	3,603,057,162	1.517
PaddedItem (own cacheline)	2,398,923,278	2,912,323,923	1.214

Cache metrics:

Mode	L1D Miss (LD)	L1D Miss (ST)	L1D TLB Miss	L2 TLB Miss
Item (shared cacheline)	266,944,790	3,277,676	566,980	12,283
PaddedItem (own cacheline)	75,144,578	5,740,164	802,912	17,275

Mac's xctrace data makes the mechanism visible. `L1D_CACHE_MISS_LD` drops from 266.9M (`Item`) to 75.1M (`PaddedItem`), a 3.55x reduction, despite `Item`'s queue being 32x smaller (4 KB vs 128 KiB). The misses are not from working-set overflow; they come from producer writes invalidating the consumer's L1 copies. Padding each slot to its own 128-byte cache line eliminates the conflict.

Another reason Mac does much better than Linux is due to our shared L2 Cache. The Performance Core group shares an L2 Cache. Linux has to bump to the L3 cache for data to be shared; Mac only needs to go to L2.

`PaddedItem`'s IPC is lower (1.214 vs 1.517). The 128 KiB queue fills the M3 Pro P-core L1d (128 KiB) to capacity, leaving less room for control-path data. The throughput advantage (+12.5%) holds because the eliminated cross-core invalidations outweigh the L1 capacity pressure, at this queue depth. At a larger slot count the 128 KiB padded footprint would spill further past L1 and the trade could flip; not measured here.

What the Hardware Docs Say

Intel. The SDM Vol. 3A §8 defines x86-TSO: loads are not reordered with other loads, stores are not reordered with other stores, and the one reordering the architecture permits is a store passing a later load (StoreLoad). That contract is why `memory_order_acquire/release` compiled to plain `mov` in every hot path here, and why only `seq_cst` pays for an explicit fence or locked operation on x86. The Optimization Reference Manual covers `pause`, the spin-loop de-pipelining that shows up in this data as Linux's 0.052 IPC.

ARM. The Architecture Reference Manual defines a weakly ordered baseline with three explicit barriers: `DMB` (ordering), `DSB` (completion), and `ISB` (context synchronization). None of them appear in these binaries. Disassembly showed all ordering carried by the one-sided instructions themselves, `ldapr` (RCpc load-acquire, ARMv8.3) and `stlr`

(store-release), which is the mapping compilers use for C++ acquire/release on ARMv8.3+ hardware.

Apple. There is no published microarchitecture manual for Apple Silicon. No cache topology document, no coherence protocol description, no optimization guide. Every Apple-internals claim in this post is therefore one of three things, labeled where it appears: queried directly (`sysctl` line size and cache sizes), measured (timing, PMU counters, disassembly), or inferred. For inference the best available public source is Dougall Johnson's reverse-engineering of Apple cores, which is community work, not a spec, and cited as such.

Takeaways for Systems Engineers

- **Results don't transfer across ISAs, and sometimes they invert.** The same C++ produced a 26.1x V1 to V4 gain on the M3 Pro and 51.5x on the i9-10900, and the Mode 2 padding change was ~1.9% faster on Linux but 8.3% slower on Mac. Benchmark on the hardware you'll deploy on.
- **Verify the instructions you think you're benchmarking.** I assumed `fetch_add` compiled to an `ldxr/stxr` retry loop. Disassembly showed a single `ldadd`, and that correction rewrote three separate mechanism explanations. `objdump` costs one command.
- **Know what your spin hint actually does.** `_mm_pause` stalled the x86 front-end to 0.052 IPC. ARM's `yield` left the M3's out-of-order engine running the poll loop at 1.69 IPC. Same source code, opposite pipeline behavior.
- **Core placement control shows up in the tail, not the median.** On isolated Linux cores (`isolcpus` + `nohz_full` + `rcu_nocbs`), pingpong p99.9 is 454 cycles, 1.26x the median. On Mac, with no hard affinity API, p99.9 runs 3x the median (250 ns vs 83 ns, tick-quantized). Same primitive, different tail discipline.